

C o m m a n d

Command Pattern

Turn a request into an object

DEFINITION

Encapsulate a request as an object, letting you parameterize, queue, log, and undo operations.

Command reifies a request: the verb "do X" becomes a first-class object carrying `execute()`, and often `undo()`. An Invoker triggers it without knowing the Receiver that does the real work, so the action can be stored, queued, logged, and replayed like any other value.

WHY IT MATTERS

When a button calls a service method directly, the trigger is welded to the work and the action cannot outlive the call — no undo, no queue, no audit log. Reifying the request decouples invoker from receiver and makes the action storable: a history stack gives undo, a queue gives async jobs, a log gives replay. The cost is a small class (or closure) per action.

— How it works

Command	Declares execute() and often undo(); binds its receiver and arguments.
Invoker	Holds and triggers commands — knows nothing of the receiver.
Receiver	Knows how to do the real work the command calls.

HOW TO APPLY

1. Wrap the action in an object with execute() (add undo() if you need reversal).
2. Bind the receiver and arguments when you build the command, not when you run it.
3. Have the Invoker trigger commands and push executed ones onto a history.
4. For undo, store enough state (or a Memento) to reverse; pop and call undo().

LITMUS TEST

Does the action need to outlive the call — to be undone, queued, logged, or composed? If you just need to call a function now, a plain callback is enough.

— Smell → fix

The editor calls document work directly, so there is no way to undo it:

```
button.onClick = () => {  
  doc.append(text) // happens now, irreversibly  
}  
// no history, no undo, no queue
```

↓ REIFY THE ACTION AS A COMMAND

```
interface Command { execute(): void; undo(): void }  
const addText = (doc, s): Command => ({  
  execute: () => doc.append(s),  
  undo: () => doc.trim(s.length),  
})  
const history: Command[] = []  
function run(c: Command) { c.execute(); history.push(c) }  
function undo() { history.pop()?.undo() }
```

The action is now a value: `run()` executes and records it, `undo()` reverses the last one. The same command objects can be queued for a worker or logged for replay — the invoker never learns what they do.

USE WHEN

- ✓ Undo/redo
- ✓ Task queues and jobs; macro recording

AVOID WHEN

- ▲ Trivial direct calls — a needless object

— Trade-offs & pitfalls

PROS

- + Decouples invoker from work
- + Enables undo, queue, log

CONS

- Many small command classes
- Indirection

COMMON PITFALLS

- ▲ Undo needs reversal state: store enough (or a Memento) — recomputing the previous value is not always possible.
- ▲ Queued or logged commands may run twice — design execute() to be idempotent or replay-safe.
- ▲ Fat-invoker smell: if the invoker reaches into a command's internals, the decoupling is lost. Closures rarely serialize, so queued commands need plain data.

WHERE YOU SEE IT

- ▶ Undo/redo stacks in editors; Redux actions; CQRS write commands routed to one handler.
- ▶ Job / task queues (BullMQ, Sidekiq) — a serialized job is a command run later by a worker.
- ▶ GUI menu items and buttons firing command objects; database WAL / event sourcing for replay.

SEE ALSO

Memento	captures the state a command needs to undo itself
Chain of Responsibility	transports commands; a request can be routed to the handler that runs one
Strategy	both wrap behavior in an object — Command reifies a request, Strategy swaps an algorithm
Prototype	clone a command to record an independent entry in a history or log

— Interview & sources

Command vs Strategy?

Both wrap behavior in an object; Command reifies a request you can queue or undo, Strategy swaps an interchangeable algorithm.

How does undo work?

Each command stores what it needs to reverse (or a Memento); a history stack pops and calls undo().

Is a job queue Command?

Yes — a serialized job is a ConcreteCommand executed later by a worker (the invoker).

Command vs CQRS command?

Same spirit: a command object captures write intent; CQRS routes it to a single handler.

REMEMBER

Turn a request into an object with `execute()` / `undo()` — now you can pass, queue, log, and reverse it.

SOURCES

GoF — "Design Patterns" (1994): Command (behavioral)

Freeman & Robson — "Head First Design Patterns" (2nd ed., 2020): Command